

CLIP

参考:

<https://arxiv.org/pdf/2103.00020>

<https://github.com/openai/CLIP>

<https://github.com/bubbliiiing/clip-pytorch>

<https://habr.com/en/articles/537334/>

<https://openai.com/index/clip/>

https://cdn.openai.com/papers/Learning_Transferable_Visual_Models_From_Natural_Language_Supervision.pdf

<https://baijiahao.baidu.com/s?id=1847928965337015404&wfr=spider&for=pc>

https://blog.csdn.net/weixin_44791964/article/details/129941386

CLIP(Contrastive Language-Image Pre-Training)模型属于多模态预训练神经网络。该模型的核心思想是使用大量图像和文本的配对数据进行预训练，以学习图像和文本之间的对齐关系。在图像分类、图像检索、文本生成、多模态搜索等领域中均有比较强大的应用场景；

CLIP 模型主要包括两个子模型结构构成：

- 图像编码器(ImageEncoder): 负责将图像转换为特征向量，可以采用卷积神经网络(eg: ResNet 等)或者 Transformer 图像模型(eg: ViT 等)；
- 文本编码器(TextEncoder): 负责将文本转换为特征向量，一般为 Transformer 结构的文本模型(eg: Bert 等)

这两个编码器通过共享一个向量空间来实现跨模态的信息交互融合，简单来讲就是将具有相同语义的文本和图像转换成"距离"近的特征向量(共同的语义空间中)。

Mermaid

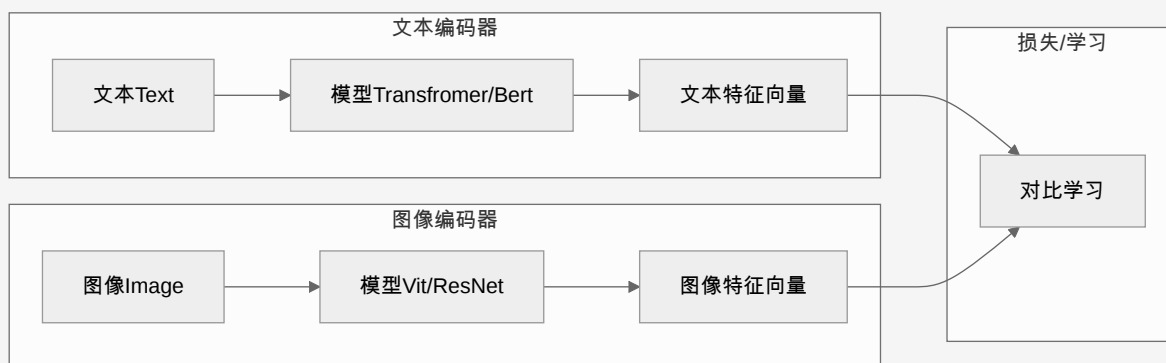
```
graph LR
    subgraph 文本编码器
        A[文本Text] --> B[模型Transformer/Bert]
        B --> C[文本特征向量]
    end
    end
```

```

subgraph 图像编码器
  D[图像Image] --> E[模型Vit/ResNet]
  E --> F[图像特征向量]
end

subgraph 损失/学习
  C --> G[对比学习]
  F --> G
end

```



CLIP 采用"对比学习"来更新模型参数，其核心思想就是通过将同一组的数据中的不同样本进行比较，从而学习样本之间的相似度和差异性，在 CLIP 模型中，主要包括下面三个阶段的操作（训练 + 使用）：

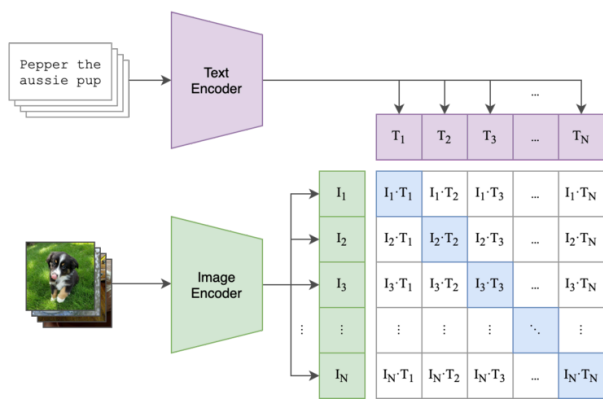
1. Contrastive pre-training：预训练阶段，使用**图片-文本对**进行对比学习训练；主要目的是为了让模型能够捕捉到图像和文本之间的深层语义联系，实现跨模态的理解；

N 张图像和 N 个文本分别通过图像编码器和文本编码器转换为对应的高阶特征向量，然后构建图像和文本高阶特征向量之间的相似度矩阵[N,N]，其实也就是两个特征向量的内积。在训练过程中，对角线上的内积代表匹配的图文对。最终损失就是通过让匹配的图像内积尽可能的大，让不匹配的图文内积尽可能的小。

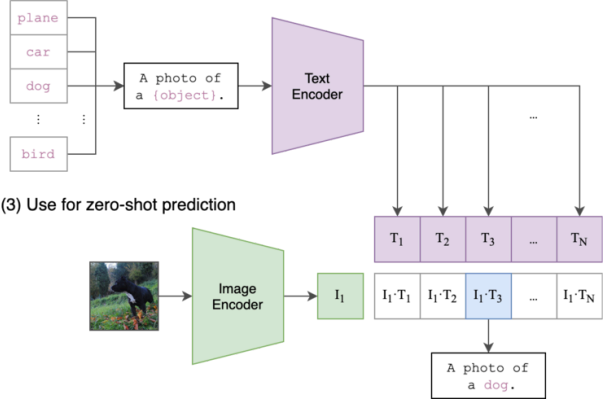
在训练过程中，同时从图像和文本两个方向来进行学习：图像到文本 和 文本到图像，这样可以保证图像、文本两个模型都可以学习到有效的特征信息；

2. Create dataset classifier from label text：提取预测类别文本的特征；
3. Use for zero-shot prediction：进行 Zero-Shot 推理预测

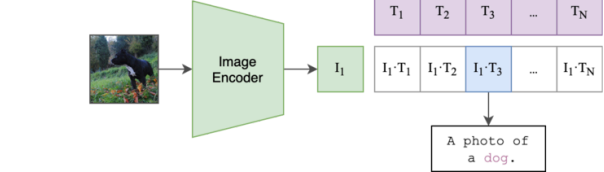
(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



▼ 预训练伪代码

Markdown

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter
# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i] 获取图像特征向量
T_f = text_encoder(T) #[n, d_t] 获取文本特征向量
# joint multimodal embedding [n, d_e] 特征向量归一化处理
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)
# scaled pairwise cosine similarities [n, n] 图像&文本特征向量计算余弦相似度
logits = np.dot(I_e, T_e.T) * np.exp(t)
# symmetric loss function 分别针对图像、文本计算损失
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
```

```
loss = (loss_i + loss_t)/2
```

其它 CLIP 体系的模型：

ALIGN:

<https://arxiv.org/pdf/2102.05918>

采用 18 亿对图像和替代文本来训练模型，期望通过增加训练数据的绝对规模来弥补 CLIP 的高质量数据构建要求。

K-LITE:

<https://arxiv.org/pdf/2204.09222>

和 ALIGN 一样，主要是为了解决对比训练过程中高质量图像-文本对数据有限的问题；通过将定义或者描述作为上下文来扩展广义的理解。

OpenCLIP:

<https://arxiv.org/abs/2212.07143>

https://github.com/mlfoundations/open_clip

主要试图理解下游任务的性能如何随数据量、计算了和模型参数数量的变化而变化，类似 Transformer 模型通过简单的缩放定律来描述模型效果一样，探索 CLIP 模型的冥等性(通过探索不同尺度、不同数据集下的模型效果)；

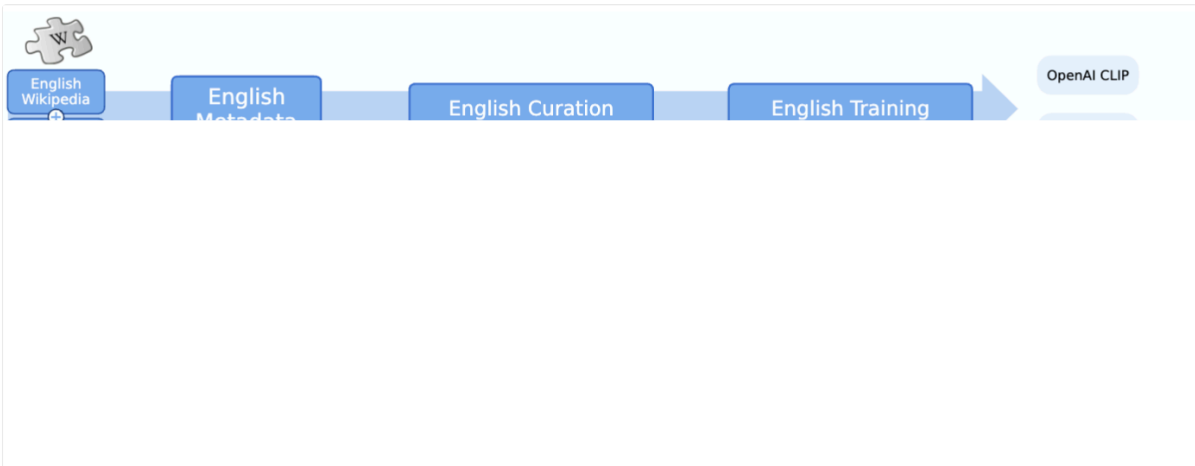
MetaCLIP:

<https://arxiv.org/abs/2309.16671>

<https://github.com/facebookresearch/MetaCLIP>

<https://www.arxiv.org/pdf/2507.22062>

优化数据管理 — CLIP 的成功在于数据而不是模型或者预训练目标；



▼ MetaCLIP2 数据预处理伪代码

Python

```
"""
Input:
D(list) raw (image, text) pairs:
    each text is assigned with a language "text.lang" by LID;
    原始图像信息以及对应的文本信息
M(dict) worldwide metadata:
    key->language code; value(list)->metadata for that language;
    全世界语言元数据, key为语言的唯一id, value为该语言对应的元数据信息
t_en(int)
    English threshold on counts of head/tail entry cutoff:
    OpenAI CLIP->20k, Meta CLIP->170k
    英文数据截断的阈值, 原始的CLIP中使用的是20K, 在MetaClipV1版本中使用的是170K

Output:
D_star(list): curated image-text pairs;
    处理后的数据结果
"""

# helper functions to compute t for each language.
# 基于给定的英文阈值t计算每个语言的对应阈值t: 先基于en_t计算p, 然后再基于p计算每个语言的t
def t_to_p(t, entry_count):
    return entry_count[entry_count < t].sum() / entry_count.sum()

def p_to_t(p, entry_count):
    sorted_count = np.sort(entry_count)
    cumsum_count = np.cumsum(sorted_count)
    cumsum_prob = cumsum_count / sorted_count.sum()
    return sorted_count[(np.abs(cumsum_prob - p)).argmin()]

# Stage 1: sub-string matching. 计算匹配的数目(各个不同语言的匹配数目)
entry_counts = {lang: np.zeros(len(M[lang])) for lang in M}
for image, text in D:
    # call substr_match which returns matched entry ids.
    text.matched_entry_ids = substr_match(text, M[text.lang])
    entry_counts[text.lang][text.matched_entry_ids] += 1
```

```

# Stage 2: compute t for each language. 针对每种语言计算对应的阈值t
p = t_to_p(t_en, entry_counts["en"]) # 基于t_en阈值计算对应的英文占比(p)
t = {}
for lang in entry_counts:
    # 保证占比p一致的情况下，计算出每个语言对应的t阈值
    t[lang] = p_to_t(p, entry_counts[lang])

# Stage 3: balancing via indepenent sampling per language.
# 针对每种语言，独立采样，保证数据平衡
entry_probs = {} # 每种语言的采样比例
for lang in entry_counts:
    entry_counts[lang][entry_counts[lang] < t[lang]] = t[lang]
    entry_probs[lang] = t[lang] / entry_counts[lang]

# 基于采样比例进行数据抽取
D_star = []
for image, text in D:
    for entry_id in text.matched_entry_ids:
        if random.random() < entry_probs[text.lang][entry_id]:
            D_star.append((image, text))
            break

```

DFN:

<https://arxiv.org/abs/2309.17425>

增加数据质量可以增加 CLIP 模型的效果；

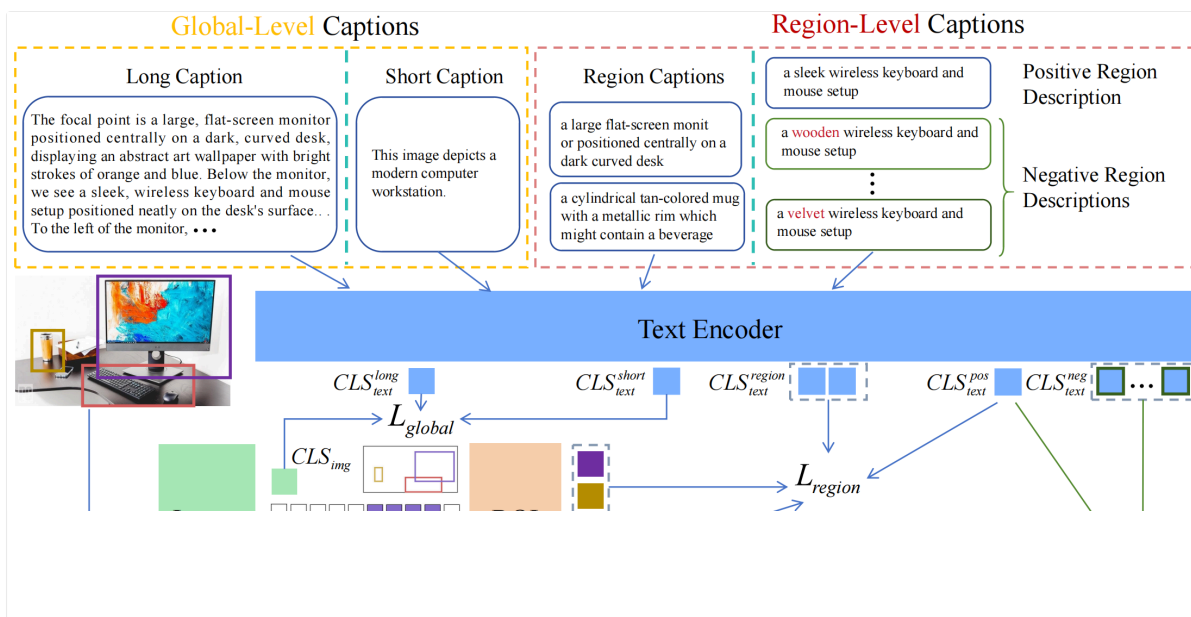
FG-CLIP 2:

<https://arxiv.org/pdf/2510.10921>

<https://github.com/360CVGroup/FG-CLIP>

<https://360cvgroup.github.io/FG-CLIP/>

增强 CLIP 模型的局部空间感知能力(两阶段训练方式 + 五维损失) + 高质量数据



FG-CLIP 训练步骤

Stage 1

Long Captions

CLS_{text}^{long}

L_{global}

Data-Adaptive Resolution



FG2-CLIP 训练步骤

